Student 23 **PATAN MOHAMMED RIYAZ KHAN** 192572126RC 8 / 10 submitted

Q1. Selection Sort

Score: 10.00 TC: 3/3 passed

CODE

```
n = list(map(int, input().split()))
for i in range(len(n)):
    min = i
    for j in range(i+1, len(n)):
        if n[j] < n[min]:
            min = j
    n[i], n[min] = n[min], n[i]
print(n)
```

INPUT `4 2 8 6 1`

OUTPUT `[1, 2, 4, 6, 8]`

Q2. Insertion Sort

Score: 10.00 TC: 3/3 passed

CODE

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

if __name__ == "__main__":
    try:
        data = list(map(int, input().split()))
        sorted_list = insertion_sort(data)
        print(sorted_list)
    except EOFError:
        pass
```

INPUT `8 5 7 1 9 3`

OUTPUT `[1, 3, 5, 7, 8, 9]`

SIMATS Engineering • CSE • CSA0802 — Python Programming • 16-08-2026 • Category: Class Work (11.8.26) • Student Submissions Report

Q3. Merge Sort

Score: 10.00 TC: 3/3 passed

CODE

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result
arr = list(map(int, input().split()))
sorted_arr = merge_sort(arr)
print(sorted_arr)
```

INPUT
```
38 27 43 3 9 82 10
```

OUTPUT
```
[3, 9, 10, 27, 38, 43, 82]
```

Q4. Diagonal Matrix

Score: 10.00 TC: 3/3 passed

CODE

```
# We clean the input string by removing brackets and spaces, then split by commas
s = input().replace("[","").replace("]","").replace(" ","").split(",")
a = list(map(int, s))

# Calculate the dimension n of the square matrix
n = int(len(a)**0.5)

f = True
for i in range(n):
    for j in range(n):
        # We check the flat list using the formula: index = (row * n) + column
        if i != j and a[i * n + j] != 0:
            f = False
            break
    if not f:
        break

print("Diagonal Matrix" if f else "Not Diagonal Matrix")
```

INPUT
```
1 0 0 0 5 0 0 0 8
```

OUTPUT
```
Diagonal Matrix
```

SIMATS Engineering • CSE • CSA0802 — Python Programming • 16-08-2026 • Category: Class Work (11.8.26) • Student Submissions Report

Q5. Row and Column Sums

Score: 6.67 TC: 2/3 passed

CODE

```
def matrix(s):
    a = []
    row = []
    num = ""
    d = 0

    for ch in s:
        if ch == "[":
            d += 1
        elif ch.isdigit() or ch == "-":
            num += ch
        elif ch == ",":
            if num != "":
                row.append(int(num))
                num = ""
        elif ch == "]":
            if num != "":
                row.append(int(num))
                num = ""
            if d == 2:
                a.append(row)
                row = []
            d -= 1
    return a

s = input().replace(" ", "")

A = matrix(s)

row_sums = []
for row in A:
    row_sums.append(sum(row))

num_rows = len(A)
num_cols = len(A[0])
col_sums = []

for j in range(num_cols):
    current_col_sum = 0
    for i in range(num_rows):
        current_col_sum += A[i][j]
    col_sums.append(current_col_sum)

print(f"Row= {row_sums}")
print(f"Col= {col_sums}")
```

INPUT

```
[[5,6],[7,8]]
```

OUTPUT

```
Row= [11, 15]
Col= [12, 14]
```

SIMATS Engineering • CSE • CSA0802 — Python Programming • 16-08-2026 • Category: Class Work (11.8.26) • Student Submissions Report

Q6. Maximum Subarray Sum

Score: 6.67 TC: 2/3 passed

CODE

```
def max_subarray_sum(arr):
    best = arr[0]
    current = arr[0]
    for x in arr[1:]:
        current = max(x, current + x)
        best = max(best, current)
    return best

# Read input as space-separated integers
a = list(map(int, input().split()))
print(max_subarray_sum(a))
```

INPUT

```
-1 -2 -3
```

OUTPUT

```
-1
```

SIMATS Engineering • CSE • CSA0802 — Python Programming • 16-08-2026 • Category: Class Work (11.8.26) • Student Submissions Report

Q7. Common Elements Among Three Lists

Score: 10.00 TC: 3/3 passed

CODE

```
def parse_list(s):
    s = s.strip()
    if not s:
        return []
    return list(map(int, s.split()))

def main():
    raw = input().strip()

    # Case 1: format is "a b c;d e f;g h i"
    if ';' in raw:
        parts = raw.split(';')
        # If fewer than 3 parts are provided, pad with empty lists
        while len(parts) < 3:
            parts.append('')
        a = parse_list(parts[0])
        b = parse_list(parts[1])
        c = parse_list(parts[2])

        common = sorted(set(a) & set(b) & set(c))
        print(common)
        return

    # Case 2: numbers are given without semicolons.
    # We'll try to split into 3 equal sized lists.
    nums = list(map(int, raw.split()))
    n = len(nums)
    if n % 3 != 0:
        # If cannot split evenly, treat as no solution (safe fallback)
        print([])
        return

    k = n // 3
    a = nums[:k]
    b = nums[k:2*k]
    c = nums[2*k:]

    common = sorted(set(a) & set(b) & set(c))
    print(common)

if __name__ == "__main__":
    main()
```

INPUT

```
1 2 3
2 3 4
2 5
```

OUTPUT

```
[]
```

SIMATS Engineering • CSE • CSA0802 — Python Programming • 16-08-2026 • Category: Class Work (11.8.26) • Student Submissions Report

Q8. Matrix Multiplication

Score: 6.67 TC: 2/3 passed

CODE

```
s=input()
s=s.replace("A=","")
s=s.replace("B=","")
s=s.replace("[","")
s=s.replace("]","")
s=s.replace(","," ")
a=list(map(int,s.split()))
if len(a)==2:
    ans=a[0]*a[1]
    print("[[%d]]"%ans)
else:
    A=[[a[0],a[1]],[a[2],a[3]]]
    B=[[a[4],a[5]],[a[6],a[7]]]
    C=[[0,0],[0,0]]
    for i in range(2):
        for j in range(2):
            for k in range(2):
                C[i][j]+=A[i][k]*B[k][j]
    print("[[%d, %d]"%(C[0][0],C[0][1]))
    print("[%d, %d]]"%(C[1][0],C[1][1]))
```

INPUT

```
A=[[1,0],[0,1]] B=[[4,5],[6,7]]
```

OUTPUT

```
[[4, 5]
[6, 7]]
```

Q9. Sort Rotate Min Max

— Not Submitted —

Q10. Sort Matrix Search

— Not Submitted —